Verification Process

A sample procedure showing the volatility surface tool works as intended

Kevin Williams | October 23, 2026

1 Approach: compare against answers known in advance

A plot that looks plausible is not proof. This procedure checks the tool against answers that are known *before* it runs, using four kinds of evidence of rising strength: known formulas, round trips, a planted truth, and independent programs that share no code with MATLAB. It takes about two minutes and needs no market data for the first six checks.

Run it

In MATLAB: `verify_demo` offline checks only

In MATLAB: `verify_demo("snapshots/<file>.csv")` adds the real-data check

In a terminal: `python3 verify_independent.py snapshots/<file>.csv` independent check

2 The checks

#	Check	Why it proves something	Pass if
1	Textbook prices	A worked example (stock 100, strike 100, 1 year, 5% rate, 20% volatility) has one correct call price, put price and vega	miss $< 10^{-5}$
2	Symbolic identities	The pricing formula must satisfy the Black-Scholes equation and put-call parity; MATLAB tests both	residual $< 10^{-8}$
3	Round trip	Price $\rightarrow$ volatility must return the volatility put in	error $< 10^{-6}$
4	Impossible prices	A call worth less than intrinsic value or more than the stock must be rejected, not “solved”	NaN returned
5	Planted smile	Build a fake chain from a smile we choose, run the real pipeline, and confirm it recovers that smile at every quoted strike	error $< 10^{-5}$
6	Unseen points	Query the surface at strikes and maturities that were *not* quoted and compare with the planted truth	error $< 5 \times 10^{-3}$
7	Real snapshot	On live data, MATLAB’s volatilities must match the independent OCaml engine	max diff $< 10^{-6}$

Checks 5 and 6 are the strongest offline evidence. The planted smile is $\sigma(x) = 0.25 + 0.6x^2 - 0.15x$ with $x = \ln(K/S)$, a curved, tilted shape like a real equity smile, quoted at three expiries (30, 60 and 120 days). Because the answer is known exactly, any error is the tool’s own.

3 Reference values, computed independently

The expected numbers below were calculated outside MATLAB, with SciPy, so they do not share any code with the tool.

Textbook case (S=K=100, T=1, r=5%, σ=20%)	**Expected**	**Dividend yield 2%**
Call price	10.450584	9.227006
Put price	5.573526	6.330081
Vega	37.524035	37.901158
Put-call parity residual	0	0

4 Evidence gathered so far

Evidence	**Result**	**Where produced**
Independent Python solver (Brent root finder) vs OCaml on the AAPL snapshot of 12 Oct 2026	140 out-of-the-money contracts across 8 expiries, all solved; median difference 2.55×10^{-7}, maximum 4.99×10^{-7}	Python, measured while writing this document
MATLAB pipeline vs OCaml on the same snapshot	Same 140 contracts; median 2.6×10^{-7}, maximum 5.0×10^{-7}; no bisection fallback needed	MATLAB, earlier session
Round-trip test grid (`test_vol_pipeline`)	142 cases tested, 18 skipped where price carries no information about volatility; worst error 3.1×10^{-7}	MATLAB, earlier session
Planted-smile surface, unseen points (check 6)	Largest miss about 1.1×10^{-3} (0.1 volatility points), mean 1.5×10^{-4} across 2000 random points	Python replica of the interpolation, not MATLAB

Reading the agreement. The OCaml engine prints volatilities to six decimals, so the largest possible rounding difference is 5×10^{-7}. Both independent programs land exactly at that ceiling, which means the remaining difference is the printing, not the mathematics. Three unrelated implementations (OCaml, MATLAB, SciPy) agree on 140 real contracts.

5 Procedure, step by step

1. Open MATLAB in the repository's `matlab` folder.
2. Run `test_vol_pipeline` and `test_surface`. Each prints `[ok]` lines and ends with "All ... checks passed."
3. Run `verify_demo`. A report like the one below appears, followed by a figure comparing the planted smile with the recovered one at a 90-day maturity that was never quoted.
4. Run `verify_demo("snapshots/<file>.csv")` on a saved snapshot to add check 7.

5. Run `verify_independent.py` on the same snapshot and confirm the Python numbers sit near the MATLAB ones.

6. Record the outcome in the sign-off sheet below.

Shape of the report (layout shown; numbers will be your own):

```
========== VERIFICATION REPORT ==========
[PASS] Textbook call, put and vega     call 10.450584, put 5.573526, ...
[PASS] Black-Scholes PDE holds         largest residual ...
[PASS] Put-call parity holds           largest residual ...
[PASS] Price to volatility round trip  ... worst error ...
[PASS] Impossible prices return NaN    both rejected
[PASS] Planted smile recovered at ...  ... worst error ...
[PASS] Surface accurate at points ...  400 points, worst error ...
-----------------------------------------
ALL 7 CHECKS PASSED
```

6 If a check fails

Failing check	Most likely cause
1 or 2	The pricing formula or its symbolic derivation changed (a sign or discount-factor error)
3	The solver's bracket or tolerance changed, or the vega used for the Newton step is wrong
4	The no-arbitrage guard was removed, so impossible prices produce numbers
5	The forward-price estimate or the out-of-the-money selection is wrong, shifting strikes
6	The interpolation changed: scaling of axes, or interpolating the wrong quantity
7	Bad data in the snapshot, or the dividend-yield or interest-rate column differs between engines

7 Sign-off sheet

Date / ticker	test_vol_pipeline	test_surface	verify_demo	Python check

8 What this does not prove

- That Yahoo's quotes are correct or current. The procedure verifies the *computation*, not the data.
- That the surface is free of arbitrage. The interpolated surface is a faithful picture of the quotes; a no-arbitrage fit is a planned later project.
- That Black-Scholes is the right model. Its disagreement with the market is precisely what the surface displays.